

Introduction à l'algorithmique et la programmation

Dr Ibrahima GAYE - UUniversité Alioune DIOP de Bambey

16 juin 2022

CHAPITRE 2 : LES STRUCTURES DE CONTRÔLE : NOTIONS FONDAMENTALES

Les structures conditionnelles

Table des matières

1	Introduction	3
2	Les structures conditionnelles	3
2.1	Présentation	3
2.2	Structure alternative	4
2.3	La structure conditionnelle au sens strict Si...Alors	6
3	Les expressions conditionnelles	7
3.1	Les comparaisons simples	7
3.2	Les comparaisons simples	8
3.3	Les variables booléennes	9
4	Un extension de la structure Si	9

1 Introduction

En programmation procédurale comme en algorithmique (qui respecte les contraintes fondamentales de la programmation!), **l'ordre des instructions est primordial**.

Le processeur exécute les instructions dans l'ordre dans lequel elles apparaissent dans le programme. On dit que l'exécution est **séquentielle**.

Une fois que le programme a fini une instruction, il passe à la suivante. Tant qu'une instruction n'est pas terminée, il attend avant de continuer. Par exemple, une instruction de saisie va attendre que l'utilisateur rentre une valeur au clavier avant de continuer.

Parfois, il est nécessaire que le processeur n'exécute pas toutes les instructions, ou encore qu'il recommence plusieurs fois les mêmes instructions. Pour cela, il faudra casser la séquence. C'est le rôle des structures de contrôle.

Il existe deux grands types de structures de contrôle :

- les **structures conditionnelles** vont permettre de n'exécuter certaines instructions que sous certaines conditions
- les **structures répétitives**, encore appelées boucles, vont permettre de répéter des instructions un certain nombre de fois, sous certaines conditions

2 Les structures conditionnelles

2.1 Présentation

Les structures conditionnelles permettent d'exécuter des instructions différentes en fonction de certaines conditions. Une condition (encore appelée expression conditionnelle ou logique) est évaluée, c'est à dire qu'elle est jugée vraie ou fausse. Si elle est vraie, un traitement (une ou plusieurs instructions) est réalisé ; si la condition est fausse, une autre instruction va être exécutée, et ensuite le programme va continuer normalement.

Il existe 2 types principaux de structures conditionnelles :

- Les structures alternatives (**SI ... Alors ... Sinon**)
- Les structures conditionnelles au sens strict (**Si ... Alors**)

2.2 Structure alternative

Dans le déroulement d'un algorithme, on doit souvent choisir entre deux actions, suivant une condition concernant la valeur de certaines données. La structure alternative va permettre d'effectuer des choix.

La syntaxe générale de cette structure est la suivante :

Si <condition> Alors

 <traitement1>

Sinon

 <traitement2>

Dans la figure ci-après, on présente l'algorithme de la structure conditionnelle alternative.

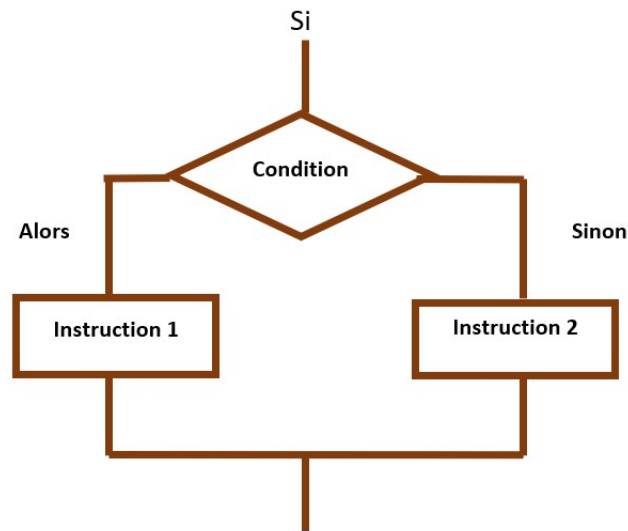


FIGURE 1 – Structure conditionnelle alternative

Pour une meilleure lisibilité du programme, on décale le **Alors** et le **Sinon** par rapport au **Si**. On peut faire apparaître un trait vertical entre **Si** et **Finsi**.

Pour l'instant cela peut paraître superflu, mais en fait quand les programmes se compliquent, ces règles d'écriture facilitent grandement leur relecture.

Exemple :

Supposons que nous ayons besoin, dans un programme, d'écrire un message précisant si la valeur d'une variable, nommée *a*, est positive ou négative.

Pour cela on va utiliser la structure alternative

ALGORITHME signe

VARIABLES n : entiers

DEBUT

 Ecrire "entrez un nombre"

 Lire n

 Si $n > 0$ Alors

 Ecrire "valeur positive"

 Sinon

 Ecrire "valeur négative ou nulle"

FIN

Si la condition $n > 0$ mentionnée après le mot **Si** est vraie, on exécute ce qui figure après le mot **Alors** ; si la condition est fausse, on exécute ce qui figure après le mot **Sinon**.

Rappelons que les traitements apparaissant après les mots **Alors** et **Sinon**, peuvent être constitués d'une instruction simple, comme dans notre premier exemple, mais aussi d'un ensemble d'instructions, appelé **bloc d'instructions**.

Exemple de structure alternative avec **bloc d'instructions**

Nous voulons un programme qui mémorise et affiche la somme ou le produit de 2 nombres, suivant le choix de l'utilisateur. Ce programme doit saisir les deux nombres voulus ainsi que la lettre représentant l'opération à effectuer. Si la lettre est s (comme somme), il calcule et affiche la somme, et si la lettre est p (ou tout autre caractère), le programme doit calculer et afficher le produit.

ALGORITHME Calculer

VARIABLES

$nb1$, $nb2$, res : entiers

op : caractère

DEBUT

 Ecrire "Entrez deux nombres"

 Lire $nb1$, $nb2$

 Ecrire "entrez la première lettre de l'opérateur voulu"

 Lire op

 Si $op = 's'$ Alors

 Debut

$res \leftarrow nb1 + nb2$

```
    Ecrire "la somme est", res
    Fin
Sinon
    Debut
    res ← nb1 + nb2
    Ecrire "la somme est", res
    Fin
FIN
```

2.3 La structure conditionnelle au sens strict Si...Alors

Cette structure est utilisée si on veut exécuter une instruction seulement si une condition est vraie et ne rien faire si la condition est fausse. Elle évite d'écrire sinon rien.

La syntaxe d'une structure conditionnelle est la suivante :

```
Si <condition> Alors
    <traitement>
```

Dans la figure ci-après, on présente l'algonigamme de la structure conditionnelle alternative.

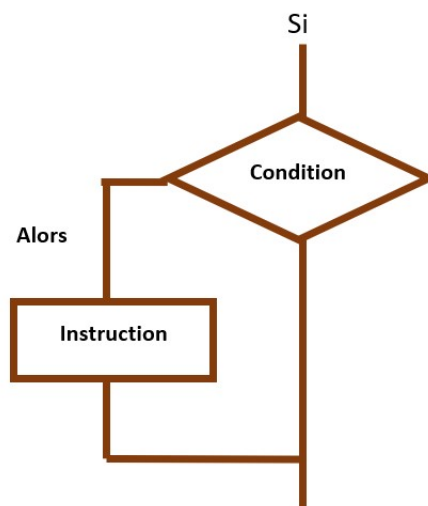


FIGURE 2 – Structure conditionnelle au sens strict

Exemple :

Dans un programme de calcul d'une facture, on veut effectuer une remise de 10% si le montant de

la facture dépasse 1000F.

Supposons que la variable qui contient le montant de la facture s'appelle mont. On veut écrire l'algorithme qui affiche le montant à payer.

Si le montant est inférieur à 1000F, on veut juste afficher le montant tel quel. Mais si le montant est supérieur à 1000F, il faut prendre en compte la remise et calculer le nouveau montant.

Le morceau d'algorithme concerné est :

Si mont > 1000 Alors

mont ← mont * 0.9

Ecrire "Net à payer : ",mont

Le programme effectue la réduction seulement si le montant est supérieur à 1000F. Sinon, il ne fait aucun traitement particulier et passe à l'instruction suivante. Dans tous les cas, le montant est affiché.

3 Les expressions conditionnelles

Une **expression conditionnelle** (ou **expression logique**, ou **expression booléenne**) est une expression dont la valeur est soit **VRAI** soit **FAUX**. Il existe plusieurs types d'expressions conditionnelles.

3.1 Les comparaisons simples

Dans nos deux exemples, les conditions que nous avons rencontré ($n > 0$) et ($op = 's'$) sont des conditions simples. Une condition simple est une comparaison de deux expressions de même type. ($n > 0$ type entier ou réel, $op = 's'$ type caractère).

Les symboles de comparaison utilisables en algorithmique sont :

$$=, \neq, <, >, \leq, \geq$$

Pour les **comparaisons de caractères**, on utilise l'**ordre ASCII**, qui respecte l'ordre alphabétique. Une lettre placée avant une autre dans l'ordre alphabétique sera inférieure à l'autre.

'a' est inférieur à 'b', mais 's' est supérieur à 'm'.

Attention, une condition simple ne veut pas dire une condition courte. Une condition simple peut être la comparaison de deux expressions comme :

$$(a + b - 3) * c \leq (5 * y - 2) / 3$$

Application

Supposons que nous voulions afficher la valeur absolue de la différence entre deux nombres entiers.

Ces nombres entiers seront notés x et y .

Nous voulons donc afficher $x - y$ si x est plus grand que y et $y - x$ sinon.

Nous écrivons pour ce faire :

Si $x > 0$ Alors

Ecrire $x - y$

Sinon

Afficher $y - x$

3.2 Les comparaisons simples

Les conditions (ou expressions conditionnelles) peuvent aussi être complexes, c'est-à-dire formées de plusieurs conditions simples ou variables booléennes reliées entre elles par les **opérateurs logiques** (ET ; OU).

Exemple :

Si $a < 0$ et $b < 0$ Alors

Instructions

Si $(a + 3 = b \text{ et } c < 0)$ ou $(a = c * 2 \text{ et } b \neq c)$ Alors

Instructions

– Et

Une condition composée de deux conditions simples reliées par et, est vraie si les deux conditions sont vraies. La condition **$a < 0$ et $b < 0$** est vraie si $a < 0$ est vraie et si $b < 0$ est vraie

– Ou

Une condition composée de deux conditions simples séparées par ou, est vraie si au moins l'une des conditions simples est vraie. La condition **$a < 0$ ou $b < 0$** est vraie si $a < 0$ ou si $b < 0$ ou si a et b sont négatifs.

– Non

Une condition précédée par non est vraie si la condition simple est fausse et inversement. **non ($a < 0$)** est vraie si $a \geq 0$

Nb : L'usage des parenthèses permet de régler d'éventuels problèmes de priorités des opérateurs

logiques.

3.3 Les variables booléennes

Les variables booléennes, comme les expressions conditionnelles, sont soit vraies, soit fausses. On peut donc affecter une expression conditionnelle à un booléen et on peut aussi trouver une variable booléenne à la place d'une expression conditionnelle.

Les variables booléennes et les expressions conditionnelles sont équivalentes. A chaque fois que l'on peut trouver une expression conditionnelle, on peut aussi trouver une variable booléenne.

Exemple :

ALGORITHME intervalles

VARIABLES

appartient : booléen

nb : réel

DEBUT

Ecrire "veuillez entrer un nombre réel"

Lire nb

appartient $\leftarrow$ (nb < 10 et nb > 5) ou (nb > 15 et nb < 20)

Si appartient Alors

Ecrire "Le nombre appartient aux intervalles définies"

Sinon

Ecrire "Le nombre n'appartient pas aux intervalles définies"

FIN

Ce programme saisit un nombre et affiche si ce nombre est compris dans les intervalles $[5, 10]$ ou $[15, 20]$.

4 Un extension de la structure Si

La structure Selon (Cas où) permet de choisir le traitement à effectuer en fonction de la valeur ou de l'intervalle de valeur d'une variable ou d'une expression. Cette structure permet de remplacer avantageusement une succession de structures Si ... Alors.

La syntaxe de cette structure est :

SELON expression FAIRE

VALEUR_1 : traitement 1

VALEUR_2 : traitement 2

VALEUR_3 : traitement 3

VALEUR_4 : traitement 4

...

SINON : traitement par défaut

FinSELON

Exemple :

Les différents cas sont des valeurs littérales. Voilà l'algorithme qui affiche le mois en toute lettre selon son numéro. Le numéro du mois est mémorisé dans la variable mois.

SELON mois FAIRE

1 : Ecrire "Janvier"

2 : Ecrire "Février"

3 : Ecrire "Mars"

4 : Ecrire "Avril"

5 : Ecrire "Mai"

6 : Ecrire "Juin"

7 : Ecrire "Juillet"

8 : Ecrire "Août"

9 : Ecrire "Septembre"

10 : Ecrire "Octobre"

11 : Ecrire "Novembre"

12 : Ecrire "Décembre"

SINON : Ecrire "Aucun mois ne correspond à la valeur entrée"

FinSELON

Exemple :

Les différents cas possibles sont décrits par des intervalles de valeur (taux de remise différent selon le montant d'achat).

SELON montant FAIRE

≤ 1000 : taux $\leftarrow 10$

> 1000 et ≤ 3000 : taux $\leftarrow 20$

> 3000 : taux $\leftarrow 30$

FinSELON

montant montant * (1-taux/100)